

Open in app ↗

Sign up

Sign in

Medium

 Search

Android



Android App Development



AndroidDev



Linux



Linux Kernel



The Android Architecture [feat. The Linux Kernel]

5 min read · Mar 4, 2025



Emmanuel Mutura™

Follow



Listen

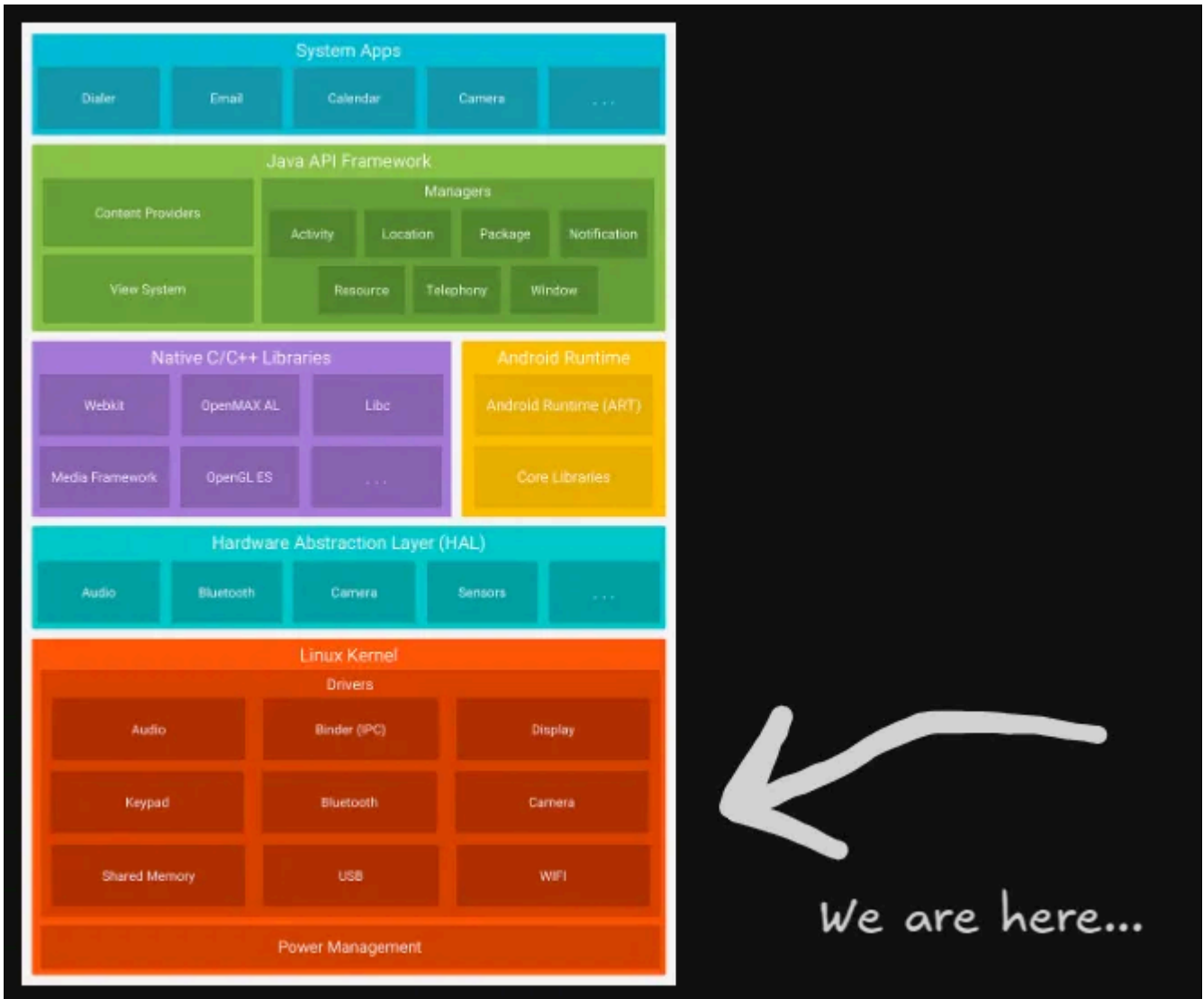


Share

If you are like me, you have probably fallen into the trap of trying to master Android stuff without going into the nooks and crannies. This, of course, results in a stammering session during interviews that you probably have PTSD about. Then again, they said that “The Devil is in the detail”. I am not sure who “they” is or how that expression is relevant here but I guess I needed to sound intelligent if I was going to take you through a deeper dive into one of the most popular topics in Android: **The Android Architecture...**

THE BACKGROUND

While a lot has been happening ever since the creation of Android, the fundamentals remain fundamental to our understanding of not just the OS itself but also the apps that we love to build for it. This article is a fun and technical explainer of the different building blocks that comprise Android. Today, we will be focusing on the deepest layer of The Android Architecture: **The Linux Kernel...**

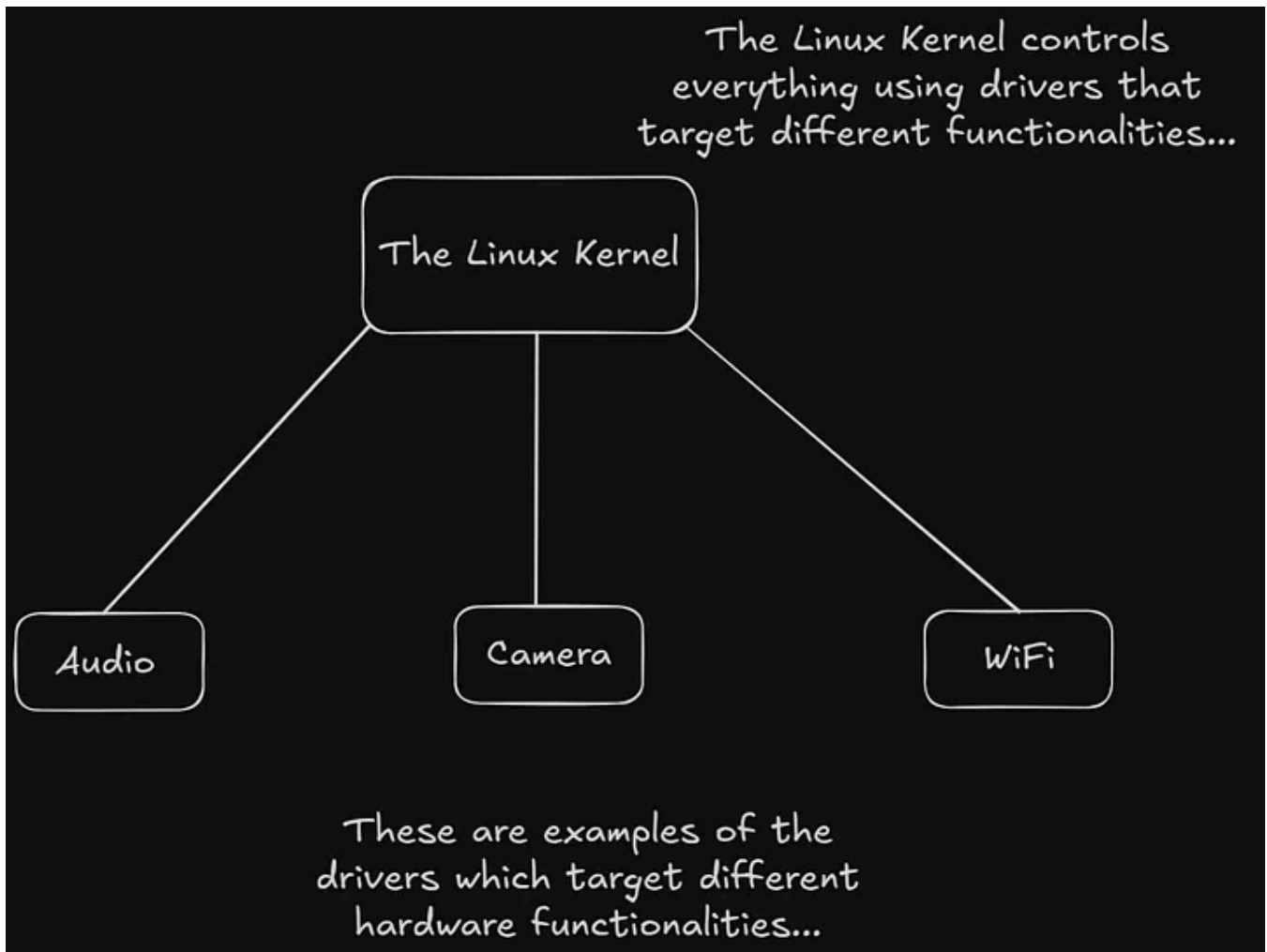


1. Overview: What is The Linux Kernel?

The Linux Kernel is the central component of Android. It acts as the core bridge between hardware and software, managing resources, scheduling tasks, and enforcing security policies. Without it, Android apps would not be able to communicate with the underlying hardware, making it the essential foundation of Android's functionality...

Analogy

Think of the Linux Kernel as the engine of your car. Just as the engine converts fuel into motion and powers every function of the vehicle, the kernel powers all operations on your Android device. It is the driving force that ensures every component works in harmony, turning user commands into real-world actions...



2. Process Management: Orchestrating the System's Pulse

Process Management in the Linux Kernel is responsible for scheduling, executing, and managing all active processes. It determines the order and duration for which each process uses the CPU, ensuring that system resources are efficiently shared among all running applications...

Analogy

Imagine that Process Management is your car's transmission system. Just as a transmission shifts gears to optimise the engine's power output based on driving conditions, the kernel's scheduler dynamically allocates CPU time to various processes. This careful management prevents "traffic jams" in processing and ensures smooth, responsive performance — much like a well-tuned car seamlessly shifting through gears during a drive...

3. Memory Management: Allocating and Optimising Resources

Memory Management in the kernel involves dynamically allocating, tracking, and reclaiming Memory for processes. Techniques like Virtual Memory, Paging, and Caching ensure that each application has the necessary space to operate without interfering with others, maintaining system stability and efficiency...

Analogy

Consider that Memory Management is the car's fuel injection and storage system. Just as fuel injectors precisely deliver the right amount of fuel to each engine cylinder and the car's compartments are organised to store essentials, the kernel allocates the appropriate amount of memory to each process. This ensures that every part of the "engine" runs at peak performance without overloading any single system...



4. Device Drivers: Bridging Hardware and Software

Device drivers are specialized modules within the kernel that facilitate communication with hardware components — such as cameras, sensors, and displays. They translate high-level commands into device-specific operations, enabling seamless integration between the software and the physical hardware...

Get Emmanuel Muturia™'s stories in your inbox

Join Medium for free to get updates from this writer.

Enter your email

Subscribe

☐ Remember me for faster sign in

Analogy

Think of device drivers as the car's onboard control modules and wiring systems. They act like the Electronic Control Units [ECUs] that interpret signals from the accelerator, brakes, and steering wheel to coordinate the car's mechanical responses. In the same way, drivers translate the kernel's commands into actions that hardware components can execute, ensuring every part of the car [or device] functions correctly...

5. File System Management: Organising Data Storage

The kernel's File System Management is tasked with organising, storing, and retrieving data on the device. It maintains the structure of directories and files, manages permissions, and ensures data integrity, allowing for fast and secure access to stored information...

Analogy

Imagine that File System Management is the car's dashboard and central computer that organises vital information. Just as the dashboard displays organised data [speed, fuel level, engine temperature, etc] and the car's compartments keep everything neatly stored, the kernel organises files and data so that everything is accessible and well-maintained. It is like having a perfectly organised trunk where every tool and accessory has its designated place...

6. Inter-Process Communication & System Calls: The Communication Network

Inter-process communication [IPC] and System Calls form the channels through which user applications interact with the kernel. These mechanisms allow apps to request services like file access, process control, or hardware operations, ensuring that the kernel can serve as a reliable intermediary between user commands and system-level actions...

Analogy

Think of IPC and System Calls as the car's internal wiring and sensor network.

When you press the accelerator or turn the steering wheel, signals travel through a network of wires to the Engine Control Unit [ECU], which then adjusts performance accordingly. Similarly, the System Calls act as the direct communication lines between applications and the kernel, ensuring that every command is transmitted accurately — just like the reliable wiring that keeps your car running smoothly...

7. Security, Customisation, and Performance: Tuning The System

The kernel incorporates robust security measures — such as SELinux — to enforce permissions and protect the system from unauthorised access. Additionally, its modular design allows for customisation and performance tuning through loadable modules and configurable settings. This flexibility enables Android Engineers to optimise the system for specific hardware or application needs...

Analogy

Imagine this aspect as the car's advanced safety systems and performance tuning options. The kernel's security features are like airbags, anti-lock brakes, and alarm systems that protect you on the road. Customisation and performance tuning are similar to upgrading your car's suspension, engine chip, or even selecting different driving modes [Eco, Sport, and Comfort] to tailor the driving experience. These features ensure that your “vehicle” runs safely, efficiently, and exactly to your preferences...

UP NEXT

The Hardware Abstraction Layer [HAL]:

<https://medium.com/@emmanuelmutura/3dbb6e96cd43>

CONCLUSION

Apart from the desire to drive your Car right now, I hope that you have now gotten a better and more technical understanding of The Linux Kernel in the context [pun intended] of The Android Architecture. Stay tuned for more content and until then, I hope that you remember what you learned today each time you go for your road trip :)....